CS 4400: Artificial Intelligence

Problem-solving agents

Contents

1. Problem-solving by searching
2. Examples
   - Road to Bucharest
   - Vacuum world
   - 8-puzzle
   - 8-queen
   - Real-world problems

Review: task environments

- PEAS description
  - Performance measure
  - Environment
  - Actuators (= effectors)
  - Sensors
- Properties of task environment
  - Partially vs fully observable
  - Stochastic vs deterministic
  - Sequential vs episodic
  - Dynamic vs static
  - Continuous vs discrete
  - Multi-agent vs single-agent

Review: agent designs

Agent designs

- Reflex agent
- Model-based reflex agent
- Goal-based agent
- Utility-based agent
- Learning agent

Goal-based agents: current best action determined by the final goal

1/2. Problem-solving by searching

Assume

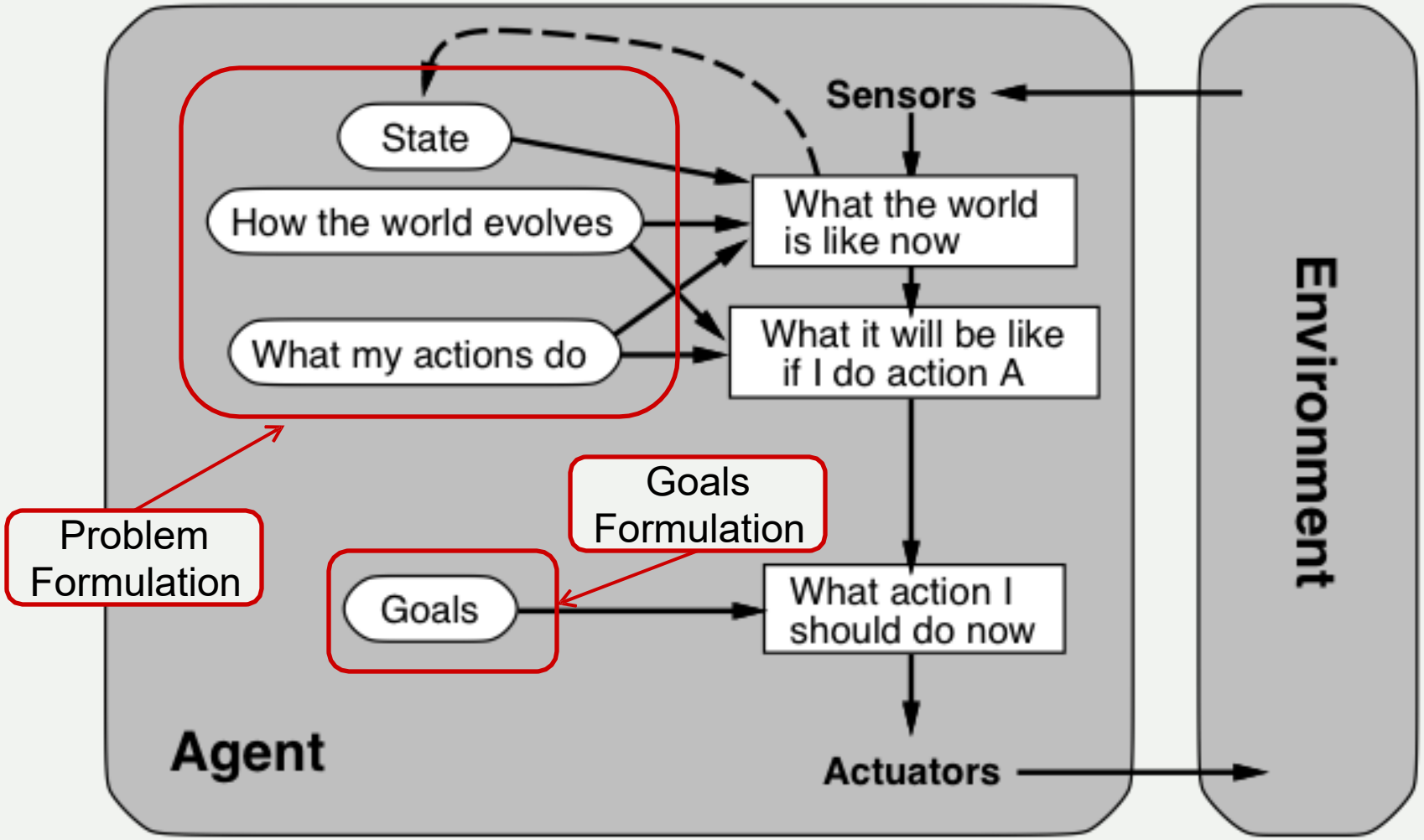
- Task environment: observable, discrete time/action/state, known, deterministic
- Agent design: goal-base agent

We can use *searching* to solve a task or a problem

Such problem-solving agents perform three main things

- Goal formulation
- Problem formulation
- Searching

Goal-based agents



Problem formulation

- Problem formulation
  - State space: all possible states + transitions
    - Initial state
    - Possible actions in state s
    - Results of an action a in state *s*
  - State space can be visualized by graph
    - Path: a sequence of states connected by sequence of actions
  - Goal test
  - Path cost

Why Search?

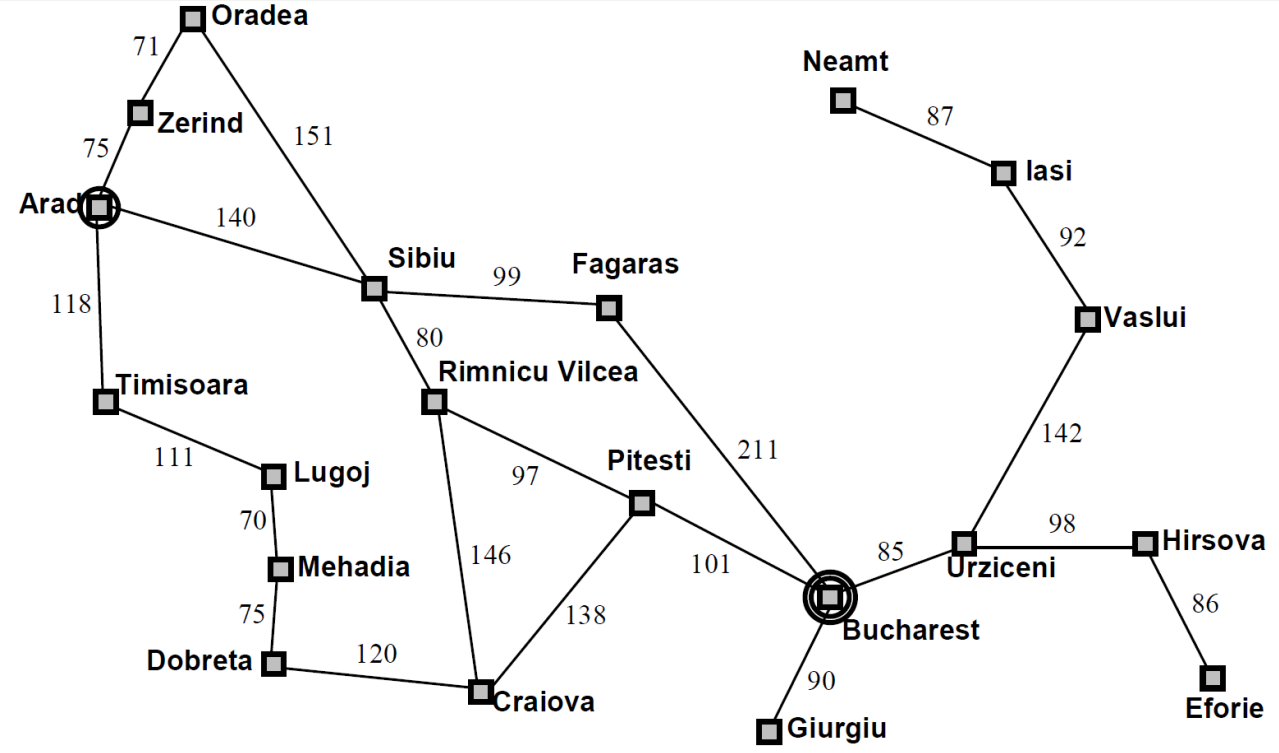
- To achieve goals or to maximize our utility, we need to predict what the result of our actions in the future will be.
- There are many sequences of actions, each with their own utility.
- We want to find, or search for, the best one.

Search Overview

- Watch this video on search algorithms:
  https://www.youtube.com/watch?v=ck-hVPDdBTk

2/2. Example: holiday in Romania

For navigation problems, the map is the state space graph itself



Task Environment: Romania

- Static / Dynamic

  Previous problem was static: no attention to changes in environment
- Observable / Partially Observable / Unobservable

  Previous problem was observable: it knew its states at all times.
- Deterministic / Stochastic

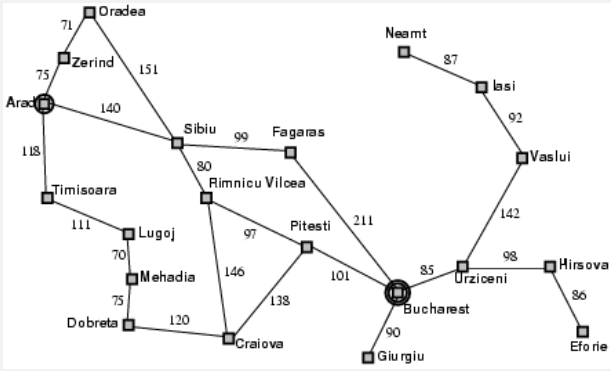
  Previous problem was deterministic: no new percepts were necessary, we can predict the future perfectly given our actions
- Discrete / continuous

  Previous problem was discrete: we can enumerate all possibilities
- Single Agent

  No other agents interacting with your cost function
- Sequential

  Decisions depend on past decisions

Example: Romania



Goal formulation: be in Bucharest

Problem formulation

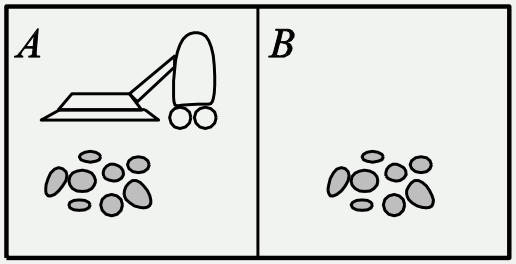
A problem is defined by four items:

initial state e.g., "at Arad“
actions set of possible actions in current state x.
transition model *Result(x,a)* = state that follows from applying action a in state x. e.g., *Result(X2 (Arad), A4 (Arad → Zerind))* = X3 (Zerind)

goal test, e.g., *x* = "at Bucharest”, *Checkmate(x)*
path cost (additive)
e.g., sum of distances, number of actions executed, etc.
c(x,a,y) is the step cost, assumed to be ≥ 0

A solution is a sequence of actions leading from the initial state to a goal state

Vacuum world



Goal formulation: clean all rooms

Problem formulation

State: agent location (A or B), dirt locations (A and/or B)

Initial state: any state

Actions: Left, Right, Suck

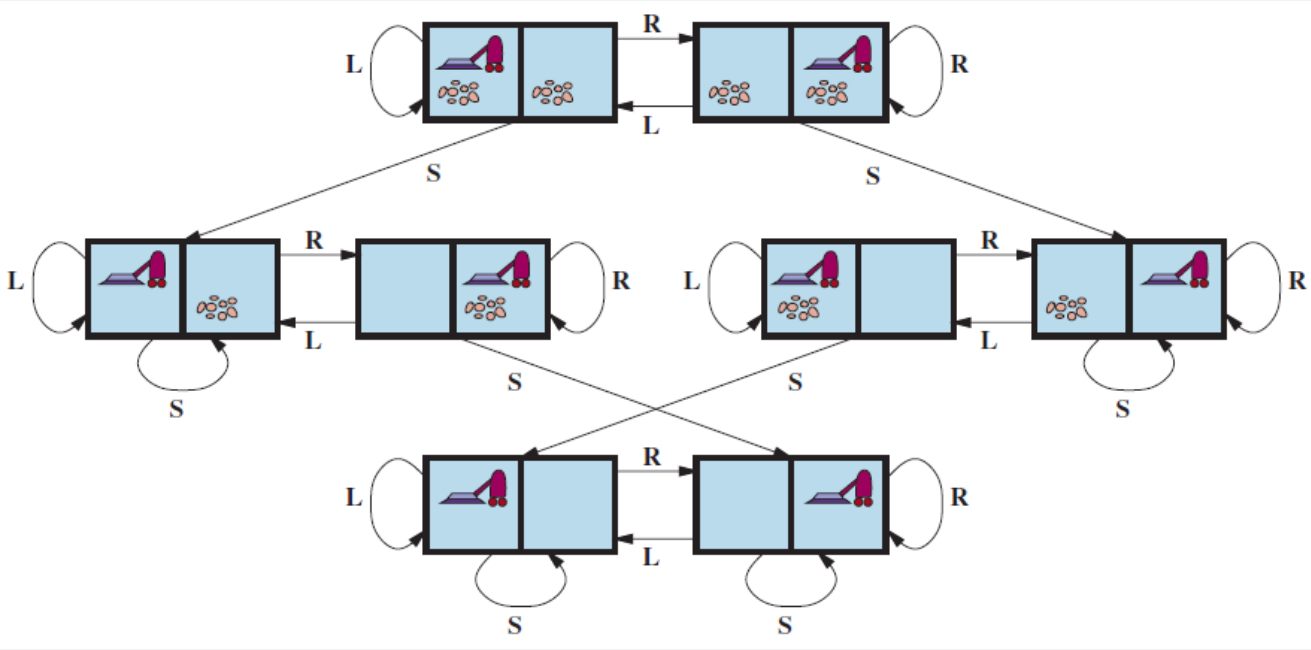
Transition: (A, Right) → B

Goal test: all squares are clean

Path cost: unit step cost (total cost = # of step)

Vacuum world

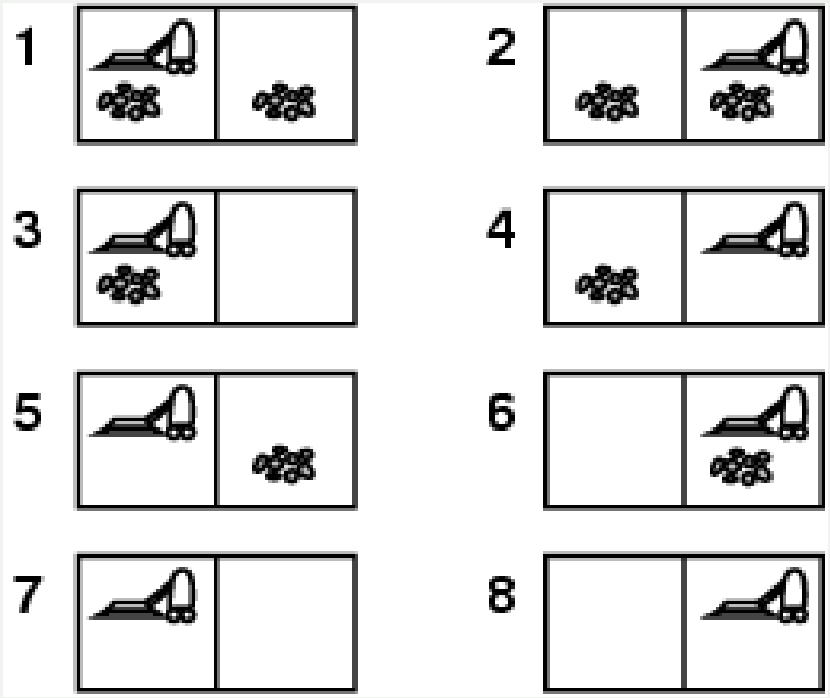
State space graph: node, edge, path



states? discrete: dirt and robot location
initial state? any
actions? *Left*, *Right*, *Suck*
goal test? no dirt at all locations
path cost? 1 per action

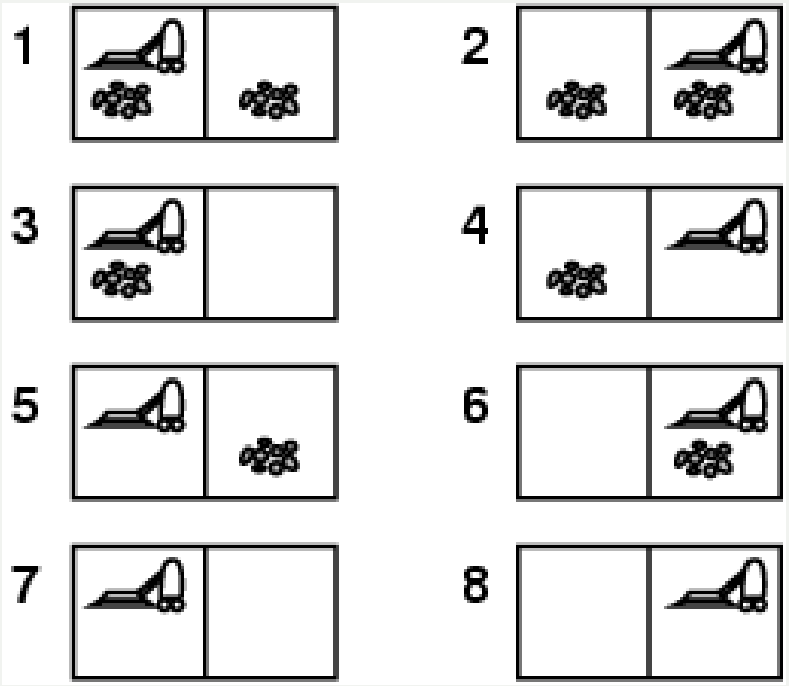
Vacuum world

- Observable, start in #5.
  Solution?



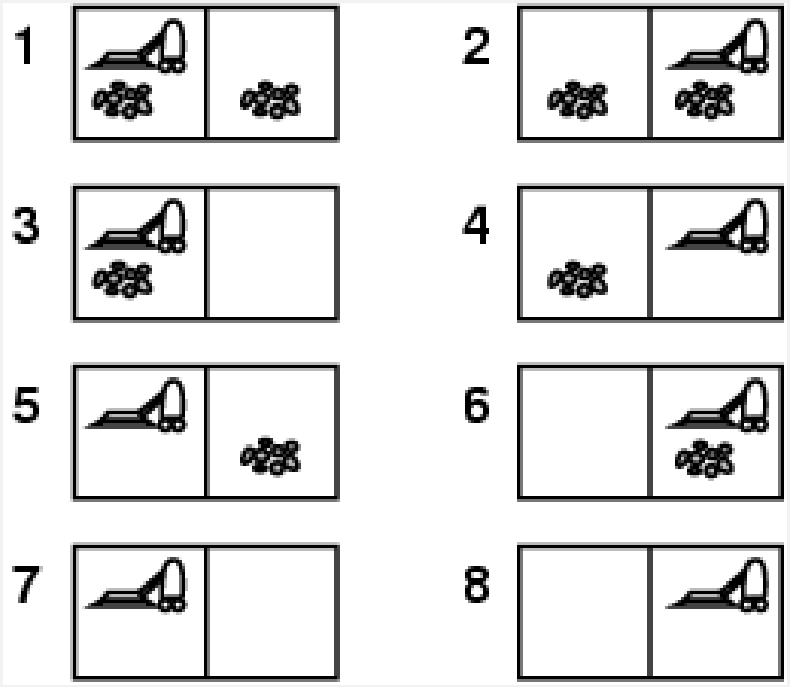
Vacuum world

- Observable, start in #5. Solution? *[Right, Suck]*
- Unobservable, start in {*1,2,3,4,5,6,7,8*} e.g., Solution?

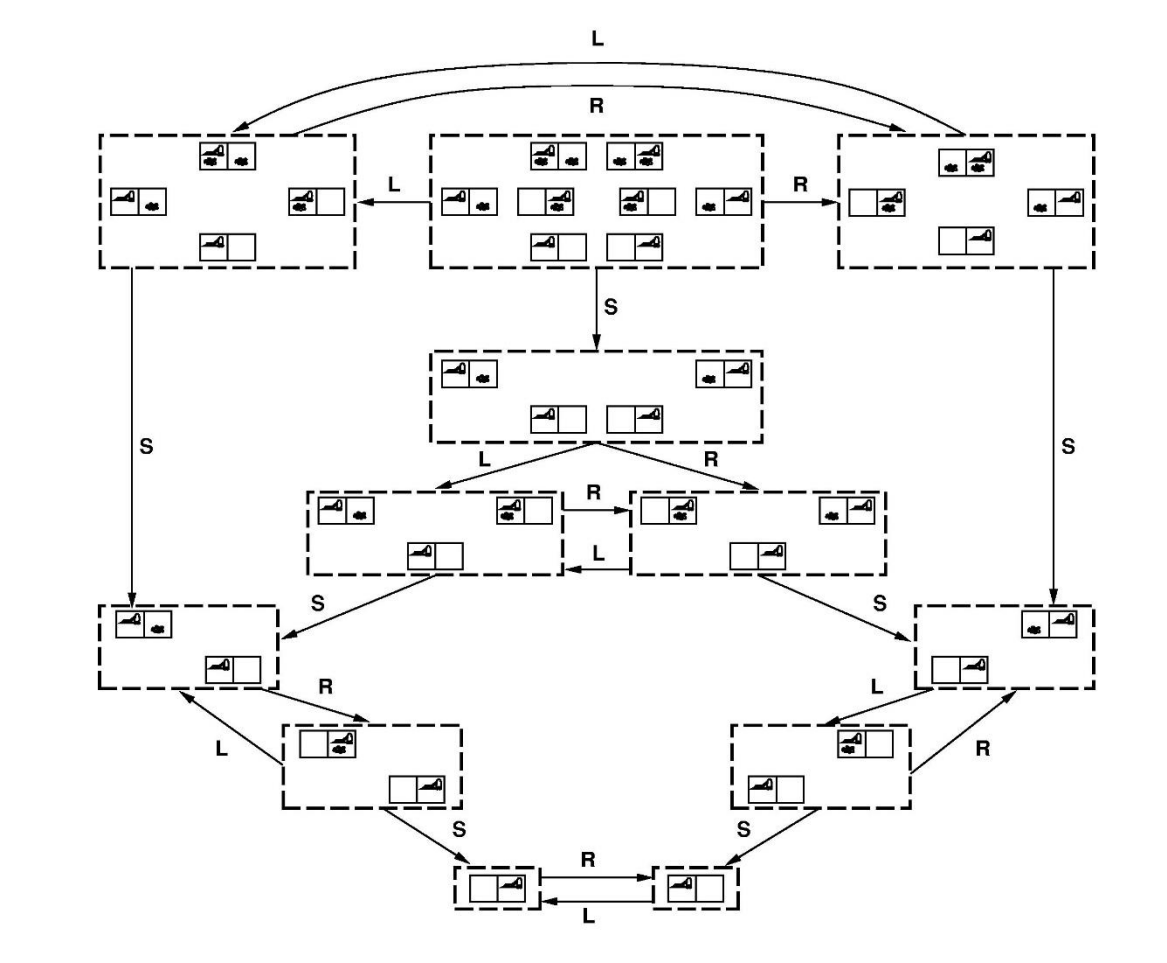


Vacuum world

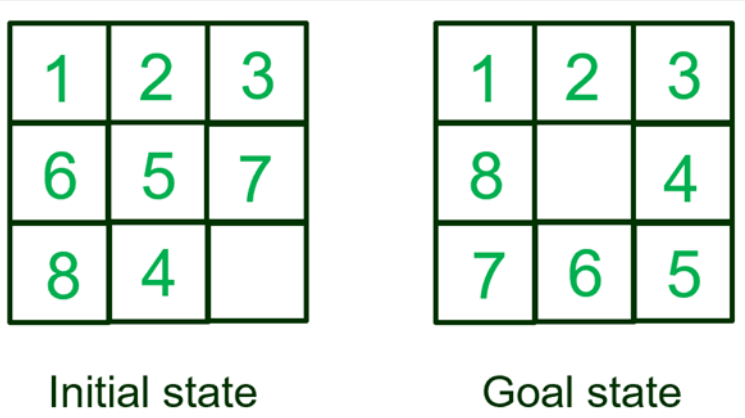
- Unobservable, start in {*1,2,3,4,5,6,7,8*} e.g., Solution?
  [Right,Suck,Left,Suck]



Vacuum world



8-puzzle



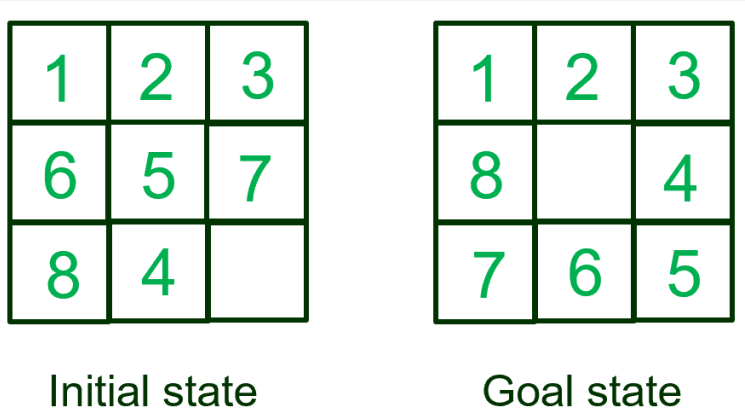
Goal: slide 8 tiles to reach the goal configuration

Problem formulation

- states?
- initial state?
- actions?
- goal test?
- path cost?

Try yourselves

8-puzzle

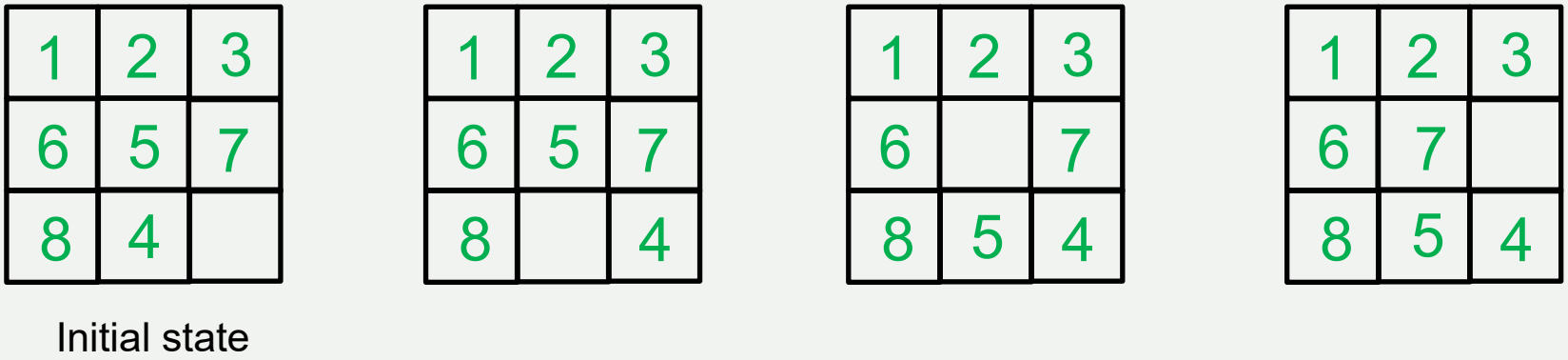


Goal: slide 8 tiles to reach the goal configuration

Problem formulation

- states? locations of tiles
- initial state? given
- actions? move blank left, right, up, down
- goal test? goal state (given)
- path cost? 1 per move

Solution to an 8-puzzle



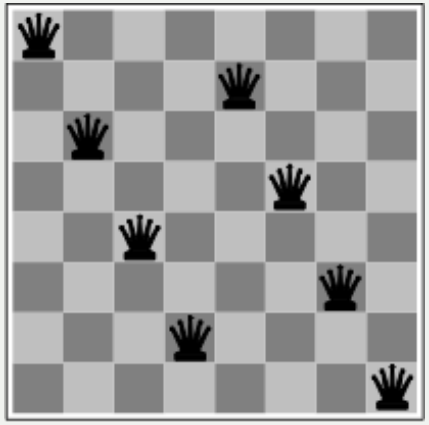
8-puzzle

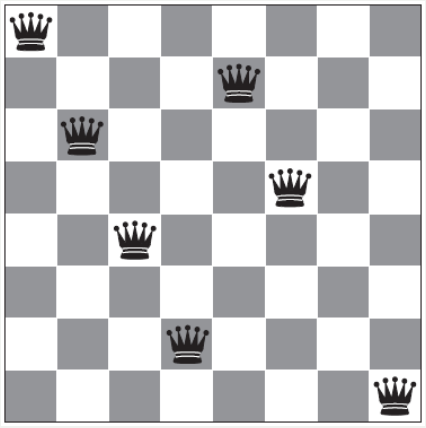
Trivial?

- 3x3 (8-puzzle): 9!/2 = 181,440 states
- 4x4 (15-puzzle): 1.3 trillion states
- 5x5 (24-puzzle): 10^{25} states
- *NP-complete

*NP-complete problem: any of a class of computational problems for which no efficient solution algorithm has been found

8-queens

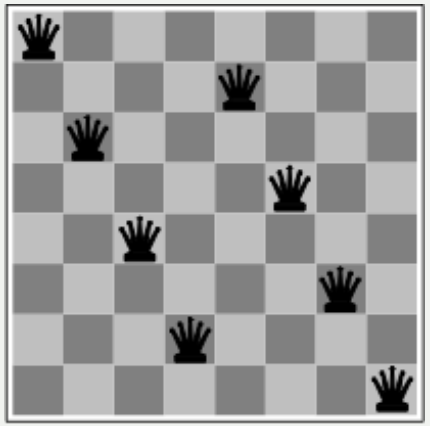




Goal: place 8 queens on 8 x 8 chessboard so that no two queens attack each other

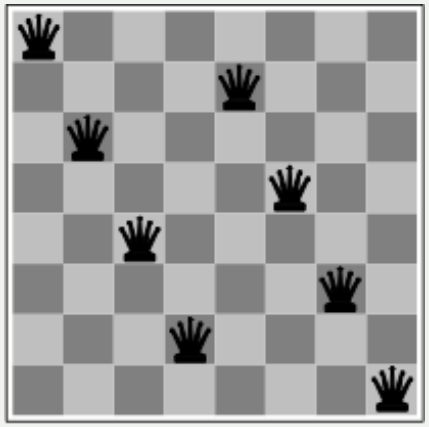
Problem formulation (see next slide)

8-queens



- states? -any arrangement of n<=8 queens
  -*or* arrangements of n<=8 queens in leftmost n columns, 1 per column, such that no queen attacks any other.
- initial state? no queens on the board
- actions? -add queen to any empty square
  -*or* add queen to leftmost empty square such that it is not attacked by other queens.
- goal test? 8 queens on the board, none attacked.
- path cost? 1 per move

8-queens



Naïve formulation

- 64 squares, 8 queens
- 64 x 63 x … x 57 = 1.8 x 10^{14} states

Better formulations

- 8^8 states
- 8! states
- 2,057 states

Problem formulation is not unique, some better than others.

Knuth's Conjecture (1964)



Donald Knuth (Stanford)

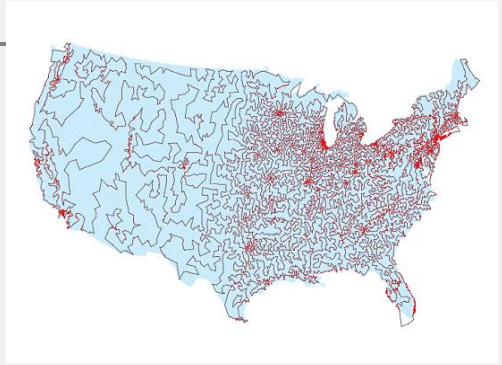
"father of the analysis of algorithms"
ACM Turing Award (1974)
National Medal of Science (1979)

Any integer > 4 can be reached from 4 via a sequence of square root, floor, and factorial operations.

$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}} \right\rfloor = 5$$

- States: positive real numbers.
- Initial state: 4.
- Goal state: the desired integer > 4.
- Actions: square root, floor, or factorial operation.
- Action cost: 1.

Industrial problems



Route-finding/navigation problem (e.g., from Ames, IA to Mountain View, CA)

Touring problem

Visit every city in the map, starting and ending in Bucharest

Traveling salesperson problem

Round trip visiting 13,509 US cities with population size > 500 exactly once

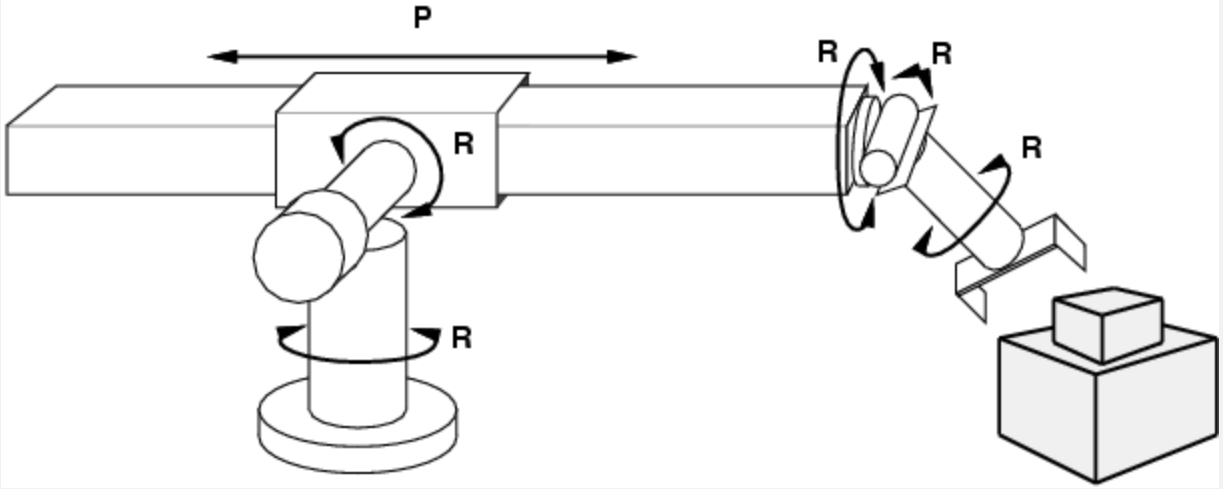
VLSI layout

layout millions of components and connections on a chip, to minimize circuit delays, stray capacitances, heat, and maximize yield

Automatic assembly sequencing

Find a order in which to assemble the parts of some objects.

Example: robotic assembly



- states?: real-valued coordinates of robot joint angles, parts of the object to be assembled
- initial state?: rest configuration
- actions?: continuous motions of robot joints
- goal test?: complete assembly
- path cost?: time to execute+energy used

Reminder

Reading (Ch 2)

Start to read Ch 3